

SQL stands for structure query language for accessing databases, we learn in SQL how to access and manipulate data.

First we know something about SQL.

What is SQL?

- SQL stands for Structured Query Language
- SQL lets you access and manipulate databases
- SQL is an ANSI (American National Standards Institute) standard

What is Structure?

A logical or mathematical model to represent any data is called structure.

What is Query?

Query is an Question which is ask through any user to system.

What is Language?

Language is a environment in which we give input and environment can process our input.

So we can say that SQL is a language by which user can query to get data or set data also.

What Can SQL do?

- SQL can execute queries against a database
- SQL can retrieve data from a database
- SQL can insert records in a database
- SQL can update records in a database
- SQL can delete records from a database
- SQL can create new databases
- SQL can create new tables in a database
- SQL can create stored procedures in a database
- SQL can create views in a database
- SQL can set permissions on tables, procedures, and views

SQL is a Standard - BUT....

Although SQL is an ANSI (American National Standards Institute) standard, there are many different versions of the SQL language.

However, to be compliant with the ANSI standard, they all support at least the major commands (such as SELECT, UPDATE, DELETE, INSERT, WHERE) in a similar manner.

Note: Most of the SQL database programs also have their own proprietary extensions in addition to the SQL standard! Like My-SQL, SQL-Server, Access, Oracle, Sybase, DB2 and other database systems.

RDBMS

RDBMS stands for Relational Database Management System.

RDBMS is the basis for SQL, and for all modern database systems like MS SQL Server, IBM DB2, Oracle, MySQL, and Microsoft Access.

The data in RDBMS is stored in database objects called tables.

Benefits of RDBMS

RDBMS stands for Relational Database Management System.

1. The amount of data redundancy in stored data can be reduced
2. No more data inconsistencies
3. Stored data can be share by a single or multiple users
4. Standards can be set and follows
5. Data integrity can be maintained. Data integrity refers to the problem of ensuring that database contains only accurate data
6. Security of data can be achieved, **i.e.** data and programs that manipulate the data are two different entities
7. In RDBMS relationship between two tables or files can be specified at the time of table creation.
8. Most of RDBMS supports Client/Server Architecture
9. Most of RDBMS supports Distributed Databases.
10. In RDBMS there are multiple level of security
 - a. Logging of O/S level
 - b. Command level(i.e. at RDBMS level)
 - c. Object level

DATABASE

A database is a collection of one or more table in the form of data or we can say that database can be defined as a collection of coherent, meaningful data. The phrase collection of coherent data needs to have a point of reference to be understood. A simple point of reference would be the example of a postal address, this would generally contain:

- ☐ A building name
- ☐ A flat number in the building
- ☐ A road name
- ☐ An area name
- ☐ A state name
- ☐ A pin code
- ☐ A country name
- ☐ We can say that multiple addresses kept together in one place, like address book, could be termed as coherent collection of data. So **address book** is a **database** and the **postal addresses** in this are **data** that fill the database.

Table

A table is a collection of related data entries and it consists of columns and rows.

Row 1 /Column1	↓ Column2	↓ Column3
→ Row 2		
→ Row 3		
→ Row 4		
→ Row 5		

Component of SQL

1. **DDL (Data Definition Language)** : It is a set of SQL commands used to create modify and delete database structure but not data. These commands are generally not used by a general user, who should be accessing the database via an application.
2. **DML (Data Manipulation Language)** : In this area of SQL that allows changing data within the database.
3. **DCL (Data Control Language)** : It is the component of SQL statements that control access to data and the database. Occasionally DCL statements are grouped with DML Statements.
4. **DQL (Data Query Language)** : It is the component of SQL statements that allow getting data from the database and imposing ordering upon it.

Examples of DDL, DML and DCL Commands

DDL: Data definition Language statements

Examples:

- ☐ **CREATE** To create object in the database
- ☐ **ALTER** Alters the structure of the database
- ☐ **DROP** Delete object from the database
- ☐ **TRUNCATE** Remove all records from a table, including all spaces allocated for the records are removed
- ☐ **COMMENT** Add comment to the data dictionary
- ☐ **GRANT** Gives user access to the data privileges
- ☐ **REVOKE** Withdraw access privileges given with the GRANT Command

DML: Data Manipulation Language statements

Examples:

- ☐ **INSERT** Insert data into a table
- ☐ **DELETE** Delete all records from the tables, the spaces for the records remain
- ☐ **UPDATE** Update existing data in the table
- ☐ **CALL** Call a PL/SQL or Java Subprogram
- ☐ **EXPLAIN PLAN** Explain access path to data
- ☐ **LOCK** Table Control concurrency

DDL: Data Control Language statements

Examples:

- ☐ **COMMIT** Save work done
- ☐ **SAVEPOINT** Identify a point in a transaction to which you can later roll back
- ☐ **ROLLBACK** Restore database to the original since the last commit
- ☐ **SET TRANSACTION** change transaction option like what rollback segment to use
- ☐ **GRANT / REVOKE** grant or take back permission to or from the oracle users

SQL **SELECT** Statement

The SELECT statement is used to select data from a database.

The result is stored in a result table, called the result-set.

SQL SELECT Syntax

```
SELECT column_name(s) FROM table_name
```

and

```
SELECT * FROM table_name
```

 **Note:** SQL is not case sensitive. SELECT is the same as select.

Tip: The asterisk (*) is a quick way of selecting all columns!

SQL **SELECT DISTINCT** Statement

In a table, some of the columns may contain duplicate values. This is not a problem, however, sometimes you will want to list only the different (distinct) values in a table.

The DISTINCT keyword can be used to return only distinct (different) values.

SQL SELECT DISTINCT Syntax

```
SELECT DISTINCT column_name(s) FROM table_name
```

SQL **WHERE** Clause

The WHERE clause is used to extract only those records that fulfill a specified criterion.

SQL WHERE Syntax

```
SELECT column_name(s) FROM table_name WHERE column_name operator value
```

SQL AND & OR Operators

The AND & OR operators are used to filter records based on more than one condition.

The AND operator displays a record if both the first condition and the second condition is true.

The OR operator displays a record if either the first condition or the second condition is true.

```
SELECT * FROM Persons WHERE condition1 AND condition 2
```

```
SELECT * FROM Persons WHERE condition1 OR condition 2
```

SQL ORDER BY Keyword

The ORDER BY keyword is used to sort the result-set.

The ORDER BY keyword is used to sort the result-set by a specified column.

The ORDER BY keyword sort the records in ascending order by default.

If you want to sort the records in a descending order, you can use the DESC keyword.

SQL ORDER BY Syntax

```
SELECT column_name(s) FROM table_name ORDER BY column_name(s) ASC|DESC
```

SQL UPDATE Statement

The UPDATE statement is used to update existing records in a table.

SQL UPDATE Syntax

```
UPDATE table_name SET column1=value, column2=value2,... WHERE column_name=value
```

Note: Notice the WHERE clause in the UPDATE syntax. The WHERE clause specifies which record or records that should be updated. If you omit the WHERE clause, all records will be updated!

SQL INSERT INTO Statement

The INSERT INTO statement is used to insert a new row in a table.

SQL INSERT INTO Syntax

It is possible to write the INSERT INTO statement in two forms.

The first form doesn't specify the column names where the data will be inserted, only their values:

```
INSERT INTO table_name VALUES (value1, value2, value3,...)
```

The second form specifies both the column names and the values to be inserted:

```
INSERT INTO table_name (column1, column2, column3,...) VALUES (value1, value2...valuen)
```

SQL DELETE Statement

The DELETE statement is used to delete rows in a table.

SQL DELETE Syntax

```
DELETE FROM table_name WHERE column_name=value
```

Note: Notice the WHERE clause in the DELETE syntax. The WHERE clause specifies which record or records that should be deleted. If you omit the WHERE clause, all records will be deleted!

SQL TOP Clause

The TOP clause is used to specify the number of records to return.

The TOP clause can be very useful on large tables with thousands of records. Returning a large number of records can impact on performance.

Note: Not all database systems support the TOP clause.

SQL Server Syntax

```
SELECT TOP number|percent column_name(s)FROM table_name
```

```
SELECT TOP number|percent * FROM table_name
```

SQL LIKE Operator

The LIKE operator is used to search for a specified pattern in a column.

SQL LIKE Syntax

```
SELECT column_name(s) FROM table_name WHERE column_name LIKE pattern
```

SQL Wildcards

SQL wildcards can be used when searching for data in a database.

SQL wildcards can substitute for one or more characters when searching for data in a database.

SQL wildcards must be used with the SQL LIKE operator.

With SQL, the following wildcards can be used:

Wildcard	Description
%	A substitute for zero or more characters
_	A substitute for exactly one character
[charlist]	Any single character in charlist
[^charlist]	Any single character not in charlist
or	
[!charlist]	

SQL IN Operator

The IN operator allows you to specify multiple values in a WHERE clause.

SQL IN Syntax

```
SELECT column_name(s) FROM table_name WHERE column_name IN (value1,value2,...)
```

SQL BETWEEN Operator

The BETWEEN operator selects a range of data between two values. The values can be numbers, text, or dates.

SQL BETWEEN Syntax

```
SELECT column_name(s) FROM table_name WHERE column_name BETWEEN value1 AND value2
```

SQL Alias

You can give a table or a column another name by using an alias. This can be a good thing to do if you have very long or complex table names or column names.

An alias name could be anything, but usually it is short.

SQL Alias Syntax for Tables

```
SELECT column_name(s) FROM table_name AS alias_name
```

SQL Alias Syntax for Columns

```
SELECT column_name AS alias_name FROM table_name
```

SQL SELECT INTO Statement

The SQL SELECT INTO statement can be used to create backup copies of tables.

The SQL SELECT INTO Statement

The SELECT INTO statement selects data from one table and inserts it into a different table.

The SELECT INTO statement is most often used to create backup copies of tables.

SQL SELECT INTO Syntax

We can select all columns into the new table:

```
SELECT * INTO new_table_name FROM old_tablename
```

SQL **CREATE DATABASE** Statement

SQL CREATE DATABASE Syntax

```
CREATE DATABASE database_name
```

SQL **CREATE TABLE** Statement

The CREATE TABLE statement is used to create a table in a database.

SQL CREATE TABLE Syntax

```
CREATE TABLE table_name  
(  
column_name1 data_type primary key,  
column_name2 data_type,  
column_name3 data_type,  
....  
)
```

SQL **NOT NULL** Constraint

The NOT NULL constraint enforces a column to NOT accept NULL values.

The NOT NULL constraint enforces a field to always contain a value. This means that you cannot insert a new record, or update a record without adding a value to this field.

```
CREATE TABLE table_name  
(  
column_name1 data_type primary key,  
column_name2 data_type not null,  
column_name3 data_type,  
....  
)
```

SQL UNIQUE Constraint

The UNIQUE constraint uniquely identifies each record in a database table.

The UNIQUE and PRIMARY KEY constraints both provide a guarantee for uniqueness for a column or set of columns.

SQL Server / Oracle / MS Access:

```
CREATE TABLE table_name
(
column_name1 data_type not null unique,
column_name2 data_type not null,
column_name3 data_type,
....
)
```

SQL PRIMARY KEY Constraint

The PRIMARY KEY constraint uniquely identifies each record in a database table.

Primary keys must contain unique values.

A primary key column cannot contain NULL values.

Each table should have a primary key, and each table can have only ONE primary key.

SQL Server / Oracle / MS Access:

```
CREATE TABLE table_name
(
column_name1 data_type primary key,
column_name2 data_type,
column_name3 data_type,
....
)
```

SQL FOREIGN KEY Constraint

A FOREIGN KEY in one table points to a PRIMARY KEY in another table.

```
CREATE TABLE table_name
(
column_name1 data_type primary key,
column_name2 data_type,
column_name3 data_type FOREIGN KEY REFERENCES Other_table_name(column_name),
....
)
```

SQL DEFAULT Constraint

The DEFAULT constraint is used to insert a default value into a column.

The default value will be added to all new records, if no other value is specified.

My SQL / SQL Server / Oracle / MS Access:

```
CREATE TABLE table_name
(
column_name1 data_type primary key,
column_name2 data_type,
column_name3 data_type DEFAULT 'value',
....
)
```

SQL DROP TABLE, and DROP DATABASE

The DROP TABLE statement is used to delete a table.

```
DROP TABLE table_name
```

The DROP DATABASE statement is used to delete a database.

```
DROP DATABASE database_name
```

The TRUNCATE TABLE Statement

What if we only want to delete the data inside the table, and not the table itself?

Then, use the TRUNCATE TABLE statement:

```
TRUNCATE TABLE table_name
```

SQL ALTER TABLE Statement

The ALTER TABLE statement is used to add, delete, or modify columns in an existing table.

To add a column in a table, use the following syntax:

```
ALTER TABLE table_name ADD column_name datatype
```

To delete a column in a table, use the following syntax (notice that some database systems don't allow deleting a column):

```
ALTER TABLE table_name DROP COLUMN column_name
```

To change the data type of a column in a table, use the following syntax:

```
ALTER TABLE table_name ALTER COLUMN column_name datatype
```

SQL Data Types

Character strings:

Data type	Description	Storage
char(n)	Fixed-length character string. Maximum 8,000 characters	n
varchar(n)	Variable-length character string. Maximum 8,000 characters	
varchar(max)	Variable-length character string. Maximum 1,073,741,824 characters	
text	Variable-length character string. Maximum 2GB of text data	

Unicode strings:

Data type	Description	Storage
nchar(n)	Fixed-length Unicode data. Maximum 4,000 characters	
nvarchar(n)	Variable-length Unicode data. Maximum 4,000 characters	
nvarchar(max)	Variable-length Unicode data. Maximum 536,870,912 characters	
ntext	Variable-length Unicode data. Maximum 2GB of text data	

Binary types:

Data type	Description	Storage
bit	Allows 0, 1, or NULL	
binary(n)	Fixed-length binary data. Maximum 8,000 bytes	
varbinary(n)	Variable-length binary data. Maximum 8,000 bytes	
varbinary(max)	Variable-length binary data. Maximum 2GB	

image	Variable-length binary data. Maximum 2GB	
-------	--	--

Number types:

Data type	Description	Storage
tinyint	Allows whole numbers from 0 to 255	1 byte
smallint	Allows whole numbers between -32,768 and 32,767	2 bytes
int	Allows whole numbers between -2,147,483,648 and 2,147,483,647	4 bytes
bigint	Allows whole numbers between -9,223,372,036,854,775,808 and 9,223,372,036,854,775,807	8 bytes
decimal(p,s)	Fixed precision and scale numbers. Allows numbers from $-10^{38} + 1$ to $10^{38} - 1$. The p parameter indicates the maximum total number of digits that can be stored (both to the left and to the right of the decimal point). p must be a value from 1 to 38. Default is 18. The s parameter indicates the maximum number of digits stored to the right of the decimal point. s must be a value from 0 to p. Default value is 0	5-17 bytes
numeric(p,s)	Fixed precision and scale numbers. Allows numbers from $-10^{38} + 1$ to $10^{38} - 1$. The p parameter indicates the maximum total number of digits that can be stored (both to the left and to the right of the decimal point). p must be a value from 1 to 38. Default is 18. The s parameter indicates the maximum number of digits stored to the right of the decimal point. s must be a value from 0 to p. Default value is 0	5-17 bytes
smallmoney	Monetary data from -214,748.3648 to 214,748.3647	4 bytes
money	Monetary data from -922,337,203,685,477.5808 to 922,337,203,685,477.5807	8 bytes
float(n)	Floating precision number data from $-1.79E + 308$ to $1.79E + 308$. The n parameter indicates whether the field should hold 4 or 8 bytes. float(24) holds a 4-byte field and float(53) holds an 8-byte field. Default value of n is 53.	4 or 8 bytes
real	Floating precision number data from $-3.40E + 38$ to $3.40E + 38$	4 bytes

Date types:

Data type	Description	Storage
datetime	From January 1, 1753 to December 31, 9999 with an accuracy of 3.33 milliseconds	8 bytes
datetime2	From January 1, 0001 to December 31, 9999 with an accuracy of 100 nanoseconds	6-8 bytes
smalldatetime	From January 1, 1900 to June 6, 2079 with an accuracy of 1 minute	4 bytes
date	Store a date only. From January 1, 0001 to December 31, 9999	3 bytes
time	Store a time only to an accuracy of 100 nanoseconds	3-5 bytes

datetimeoffset	The same as datetime2 with the addition of a time zone offset	8-10 bytes
timestamp	Stores a unique number that gets updated every time a row gets created or modified. The timestamp value is based upon an internal clock and does not correspond to real time. Each table may have only one timestamp variable	

Other data types:

Data type	Description
sql_variant	Stores up to 8,000 bytes of data of various data types, except text, ntext, and timestamp
uniqueidentifier	Stores a globally unique identifier (GUID)
xml	Stores XML formatted data. Maximum 2GB
cursor	Stores a reference to a cursor used for database operations
table	Stores a result-set for later processing



Commands to create database, table and manipulation of data

-- command for create database

create database database_name

-- command for select database1

use database_name

-- command for create table

create table table_name(column_name1 datatype **primary key**,column_name2 datatype,column_name3 datatype,.....column_nameN datatype)

-- command for insert value in table

insert into table_name **values**(value1,value2,value3,value4,value5.....valueN)

-- command for select all columns from table(* means all columns)

select * from table_name

-- command for select particular column from table

select column_name1,column_name2,column_name3 **from** table_name

-- command for select all columns from table on condition(* means all columns)

select * **from** table_name **where** condition

-- command for select particular column from table on condition

select column_name1,column_name2,column_name3 **from** table_name **where** condition

-- command for select particular column from table on condition with and operator

select column_name1,column_name2,column_name3 **from** table_name **where** condition1 **and** condition2

-- command for select particular column from table on condition with or operator

select column_name1,column_name2,column_name3 **from** table_name **where** condition1 **or** condition2

-- command for select particular column from table on condition with and operator with order by command

select column_name1,column_name2,column_name3 **from** table_name **where** condition1 **and** condition2 **order by** column_name

-- command for select particular column from table on condition with or operator with order by command

select column_name1,column_name2,column_name3 **from** table_name **where** condition1 **or** condition2 **order by** column_name

-- command for select all columns from table on condition(* means all columns) with order by command

select * **from** table_name **where** condition **order by** column_name

-- command for select particular column from table on condition with order by command

select column_name1,column_name2,column_name3 **from** table_name **where** condition **order by** column_name

-- command for select distinct value from column_name from table without condition

select distinct column_name1,column_name2,column_name3 **from** table_name

-- command for select distinct value from column_name from table on condition

select distinct column_name1,column_name2,column_name3 **from** table_name **where** condition

-- command for select distinct value from column_name from table on condition with order by command

select distinct column_name1,column_name2,column_name3 **from** table_name **where** condition **order by** column_name

-- command for select distinct value from column_name from table without condition order by
command
select distinct column_name1,column_name2,column_name3 **from** table_name **order by**
column_name

-- command for delete all columns from table(* means all columns)
delete from table_name
delete * from table_name